

# Регистр Docker

# Розуміння Docker/Container Registries

- **Docker/Container Registries** — це репозиторії для зберігання, управління та розгортання образів контейнерів. Вони виступають як **централізовані хаби**, де розробники можуть завантажувати, ділитися та отримувати контейнерні образи.
- Registries відіграють важливу роль в управлінні контейнерами, забезпечуючи **централізоване зберігання образів**. Вони сприяють ефективному розгортанню додатків, гарантують доступність необхідних образів, допомагають у **контролі версій**, **керуванні залежностями** та забезпечують **послідовність середовищ**.
- При виборі Docker/Container registry слід враховувати такі фактори:
  - **Безпека** для захисту конфіденційних даних
  - **Масштабованість**, щоб підтримувати зростаючу інфраструктуру
  - **Інтеграція** з існуючими DevOps-інструментами та робочими процесами

Оцінюючи ці критерії, організації можуть обрати registry, який найкраще відповідає їхнім вимогам та ефективно підтримує практики DevOps.

# Топ-9 популярных Docker/Container Registries

**Amazon Elastic Container Registry (ECR)**

**Azure Container Registry (ACR)**

**Docker Hub Container Registry**

**GitHub Package Registry**

**GitLab Container Registry**

**Google Artifact Registry (GAR)**

**Harbor Container Registry**

**Red Hat Quay**

**Sonatype Nexus Repository OSS**

# Amazon Elastic Container Registry (ECR)

**Amazon ECR** — це повністю керований реєстр контейнерів Docker, який дозволяє розробникам легко **зберігати, керувати та розгортати образи Docker**. Він **безшовно інтегрується** з Amazon ECS (Elastic Container Service) та EKS (Elastic Kubernetes Service), забезпечуючи надійний та безпечний спосіб управління контейнерами та їх розгортання.

## Основні функції:

- **Автоматичне сканування образів:** ECR автоматично перевіряє ваші Docker-образи на наявність вразливостей, використовуючи базу даних Common Vulnerabilities and Exposures (CVEs). Це допомагає підтримувати безпеку, виявляючи потенційні загрози до розгортання.
- **Реплікація образів між регіонами:** дозволяє автоматично копіювати контейнерні образи між кількома регіонами AWS. Це забезпечує швидше завантаження образів та підвищує доступність завдяки географічному розподілу ближче до середовища виконання вашого додатку.
- **Приватні репозиторії:** ECR дозволяє створювати приватні Docker-репозиторії, що гарантує, що ваші образи **не доступні публічно**. Це підвищує безпеку, надаючи контроль над тим, хто може завантажувати та керувати образами.

# Ціноутворення Amazon Elastic Container Registry (ECR)

## Free Tier для нових користувачів:

- 500 МБ на місяць для **приватних репозиторіїв** протягом першого року.

## Безкоштовне зберігання для публічних репозиторіїв:

- 50 ГБ на місяць **для всіх користувачів** (нових і існуючих).
- Анонімно (без AWS-акаунта) можна передавати **500 ГБ даних на місяць** з публічного репозиторію в Інтернет безкоштовно.
- При реєстрації AWS-акаунта або автентифікації в ECR доступно **5 ТБ передачі даних на місяць** з публічного репозиторію в Інтернет безкоштовно.
- **Необмежена пропускна здатність** при передачі даних з публічного репозиторію на ресурси обчислень AWS у будь-якому регіоні AWS.

## Розрахунок безкоштовного використання:

- Безкоштовний обсяг використання обчислюється **щомісяця по всіх регіонах** і автоматично враховується у вашому рахунку.
- Невикористані обсяги **не накопичуються** на наступні місяці.



COMPUTE

Free Tier 12 MONTHS FREE

Amazon Elastic Container Registry

**50 GB**

of storage for public repositories

Store and retrieve Docker images.

500 MB of storage for private repositories

5 TB of authenticated data transfer out of public repositories

500 GB of anonymous data transfer out of public repositories

^

# Azure Container Registry (ACR)

Azure Container Registry (ACR) — це керований сервіс Docker-реєстру від **Microsoft Azure**, який дозволяє зберігати та керувати контейнерними образами для будь-яких типів розгортань в Azure.

ACR підтримує стандартні команди **Docker CLI** і безперешкодно інтегрується з **Azure Kubernetes Service (AKS)**.

## ◆ Основні можливості:

- **Автоматизовані збірки та вебхуки:**

ACR дозволяє автоматично створювати Docker-образи з репозиторіїв вихідного коду (наприклад, Azure DevOps або GitHub) та надсилати сповіщення через вебхуки при подіях, як-от push або видалення образу.

- **Автоматизація завдань:**

За допомогою **ACR Tasks** можна автоматизувати процеси створення, оновлення (patching) і тестування Docker-образів у кількох середовищах. Це ідеальне рішення для реалізації **CI/CD-процесів**.

- **Геореплікація:**

У тарифному плані **ACR Premium** доступна функція геореплікації, яка дозволяє створювати копії реєстру в різних регіонах Azure. Це підвищує продуктивність і забезпечує високу доступність, розміщуючи образи ближче до користувачів.

- **Безпека та відповідність стандартам:**

Інтеграція з **Azure Active Directory** забезпечує автентифікацію користувачів, а **Azure Role-Based Access Control (RBAC)** — гнучке керування доступом.

ACR також відповідає міжнародним стандартам безпеки — **ISO, SOC, PCI**.

# Ціноутворення Azure Container Registry

- **Базовий рівень (Basic Tier):** ідеальний для невеликих розгортань, пропонує 10 ГБ сховища та коштує приблизно **5 доларів на місяць**.
- **Стандартний рівень (Standard Tier):** орієнтований на розгортання середнього масштабу, надає 100 ГБ сховища та коштує близько **20 доларів на місяць**.
- **Преміум рівень (Premium Tier):** призначений для масштабних і критично важливих для бізнесу застосунків, включає 500 ГБ сховища та додаткові функції, такі як **геореплікація**, коштує приблизно **50 доларів на місяць**.
- **Безкоштовний рівень (Free Tier)**



Container Registry

Store and manage container images across all types of Azure deployments. Containers

1 Standard tier registry with 100 GB storage and 10 webhooks

12 months

# Реєстр контейнерів Docker Hub

**Docker Hub** — це найбільша у світі бібліотека та спільнота контейнерних образів, що пропонує централізований ресурс для **пошуку, розповсюдження та керування змінами контейнерних образів**. Вона чудово інтегрується з **Docker Engine**, забезпечуючи розробникам простий спосіб публікувати та обмінюватися контейнерними образами.

## Основні можливості:

- **Велике сховище:** Docker Hub містить мільйони публічних і приватних репозиторіїв, що дозволяє користувачам співпрацювати як у межах команд, так і в ширшій спільноті Docker. Користувачі можуть знаходити **офіційні образи** від постачальників програмного забезпечення та **сертифікований контент**.
- **Автоматичні збірки:** Підтримує **автоматичну збірку та тестування** контейнерних образів із репозиторіїв GitHub і Bitbucket після кожного коміту коду, що гарантує наявність актуальних версій образів.
- **Вебхуки та сповіщення:** Дозволяє налаштовувати **вебхуки**, які запускають сповіщення або дії у відповідь на певні події в реєстрі, наприклад, оновлення або завантаження нових образів.
- **Інтеграція та сумісність:** Повністю інтегрується з **екосистемою Docker**, забезпечуючи плавний робочий процес для користувачів Docker Engine. Підтримує всі можливості **Docker Compose** та **Docker Swarm**.



# Ціноутворення Docker Hub Container Registry

- **Безкоштовний план (Free Plan):** надає **один приватний репозиторій та необмежену кількість публічних репозиторіїв**.

Підтримуються автоматичні збірки, але з деякими обмеженнями щодо часу збірки та кількості паралельних збірок.

- **План Pro (Pro Plan):** коштує **7 доларів на місяць** і включає **необмежену кількість приватних репозиторіїв**, вищий пріоритет для автоматичних збірок та **відсутність обмежень на кількість завантажень образів**.

- **Командний план (Team Plan):** призначений для команд та організацій, коштує **11 доларів за користувача на місяць**, включає **розширене управління командами**, більше паралельних збірок та **покращені інструменти для спільної роботи**.

The image shows four pricing plans for Docker Hub Container Registry. Each plan is presented in a card with a title, description, price, features list, and a call-to-action button.

| Plan     | Frequency | Price                    | Key Features                                                                                                                                                                                                                            |
|----------|-----------|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Personal | Free      | \$0                      | ✓ Docker Desktop<br>✓ Unlimited public repositories<br>✓ Docker Engine + Kubernetes<br>✓ 200 image pulls per 6 hours<br>✓ Unlimited scoped tokens                                                                                       |
| Pro      | Monthly   | \$7 per month            | Everything in Personal plus:<br>✓ Unlimited private repositories<br>✓ 5,000 image pulls per day<br>✓ 5 concurrent builds<br>✓ 300 Hub vulnerability scans                                                                               |
| Team     | Monthly   | \$11 per user* per month | Everything in Pro plus:<br>✓ Up to 100 users<br>✓ Unlimited teams<br>✓ 15 concurrent builds<br>✓ Unlimited Hub vulnerability scans<br>✓ Add users in bulk<br>✓ Audit logs                                                               |
| Business | Yearly    | \$24 per user* per month | Everything in Team plus:<br>✓ Unlimited users<br>✓ Hardened Docker Desktop<br>✓ Centralized management<br>✓ Registry Access Management<br>✓ Single Sign-On (SSO)<br>✓ SCIM user provisioning<br>✓ VDI support<br>✓ Purchase via invoice |

\*Minimum of 5 seats, billed monthly for \$35. \$11/user/month for additional seats.

Only available with an annual subscription. \*Minimum 5 seats.

# GitHub Package Registry

**GitHub Package Registry** — це сучасний сервіс управління пакетами, який дозволяє користувачам GitHub публікувати, керувати та встановлювати програмні пакети безпосередньо разом із їхнім вихідним кодом. Він підтримує кілька форматів пакетів, включаючи **Docker-контейнери**, забезпечуючи безшовну інтеграцію з робочими процесами GitHub та CI/CD пайплайнами.

## Основні можливості:

- **Інтеграція з GitHub:** Пряма інтеграція з репозиторіями GitHub. Використовуйте той самий інтерфейс GitHub для керування вихідним кодом і пакетами, що спрощує контроль доступу та зменшує складність управління залежностями проекту.
- **Підтримка різних форматів пакетів:** Окрім Docker-образів, підтримує **npm, NuGet, Maven, Gradle та RubyGems**, що робить його універсальним для проектів на різних мовах програмування та в різних середовищах.
- **Інтеграція з CI/CD:** Легко інтегрується з **GitHub Actions**, дозволяючи автоматично публікувати пакети та керувати версіями безпосередньо з робочих процесів GitHub.
- **Видимість та контроль доступу:** Надає детальні права доступу, пов'язані з обліковими записами та ролями GitHub. Пакети можна робити **публічними** для проектів з відкритим кодом або **приватними** для внутрішнього використання, відповідно до налаштувань видимості репозиторію.

# Ціноутворення GitHub Package Registry

## Публічні репозиторії:

- Безкоштовно для всіх користувачів.
- Необмежене завантаження та публікація публічних пакетів безкоштовно.

## Приватні репозиторії:

GitHub надає різні обсяги сховища та передачі даних залежно від плану підписки користувача.

- **Безкоштовний план (Free Plan):** включає **500 МБ** сховища та **1 ГБ** на місяць передачі даних поза межами **GitHub Actions**, з необмеженою передачею даних всередині **GitHub Actions**.
- **Командний план (Team Plan):** надає **2 ГБ** сховища та **10 ГБ** на місяць передачі даних поза **GitHub Actions**, з необмеженою передачею даних всередині **GitHub Actions**.
- **План Enterprise (Enterprise Plan):** пропонує **50 ГБ** сховища та **100 ГБ** на місяць передачі даних поза **GitHub Actions**, плюс необмежену передачу даних всередині **GitHub Actions**.

## Private repositories

| Plan       | Storage | Data transfer out within Actions | Data transfer out outside of Actions |                                                        |
|------------|---------|----------------------------------|--------------------------------------|--------------------------------------------------------|
| Free       | 500MB   | Unlimited                        | 1GB per month                        | Included in Pro                                        |
| Team       | 2GB     | Unlimited                        | 10GB per month                       | <div>Most Popular</div> <div>Continue with Team</div>  |
| Enterprise | 50GB    | Unlimited                        | 100GB per month                      | <div>Start a Free Trial</div> <div>Contact Sales</div> |

GitHub Packages is not available for private repos in legacy per-repository plans.

|                                                 |                         |
|-------------------------------------------------|-------------------------|
| Additional storage                              | \$0.25 USD per gigabyte |
| Additional data transfer out outside of Actions | \$0.50 USD per gigabyte |

# GitLab Container Registry

**GitLab Container Registry** — це інтегрована частина екосистеми GitLab, яка забезпечує **безпечний та приватний реєстр Docker-образів**. Цей реєстр є безкоштовним та повністю інтегрується з **GitLab CI/CD**, що робить його зручним для **зборки, тестування та розгортання Docker-контейнерів** у проектах GitLab.

## Основні можливості:

- **Безшовна інтеграція з GitLab:** Користувачі можуть легко збирати, завантажувати та отримувати образи в/з реєстру **безпосередньо у CI/CD пайплайнах GitLab**, використовуючи єдиний інтерфейс GitLab.
- **Безпека та відповідність:** Забезпечує надійні функції безпеки, такі як **сканування контейнерів, сканування залежностей** та інтеграція з **панеллю безпеки GitLab** для комплексного управління вразливостями.
- **Ефективне зберігання образів:** Використовує **систему зберігання образів на основі Docker Distribution**, що зменшує дублювання та економить місце, роблячи сховище ефективнішим.
- **Видимість та управління дозволами:** Надає **детальний контроль доступу**, прив'язаний до прав користувачів GitLab, що гарантує доступ до образів лише авторизованим користувачам.

# Google Artifact Registry

**Google Artifact Registry** — це керований сервіс від Google Cloud для зберігання та керування різними типами артефактів, включаючи **Docker-контейнери, Maven, npm, Python пакети та інші**. Він замінює попередній сервіс Container Registry і забезпечує більш гнучке управління, інтеграцію з CI/CD та контроль доступу на рівні проекту.

## Основні можливості:

- **Підтримка кількох форматів артефактів:** Крім Docker-образів, підтримує пакети для **Maven, npm, Python** та інших систем управління пакетами.
- **Інтеграція з CI/CD:** Працює з Google Cloud Build та іншими CI/CD інструментами для автоматичного збору, публікації та розгортання артефактів.
- **Контроль доступу:** Забезпечує **гнучкі права доступу на основі IAM**, дозволяючи визначати, хто може переглядати або модифікувати артефакти на рівні проектів або репозиторіїв.
- **Безпека та відповідність:** Включає **сканування образів на вразливості**, шифрування даних у спокої та під час передавання, а також інтеграцію з політиками безпеки Google Cloud.
- **Масштабованість та ефективність:** Забезпечує високодоступне та швидке зберігання артефактів з оптимізацією для великих команд і проектів.

# Harbor Container Registry

**Harbor** — це безкоштовний open-source реєстр контейнерів, який забезпечує безпечне зберігання та керування контейнерними образами. Він розроблений для середовищ із високими вимогами до безпеки та підтримує сканування образів на вразливості й управління доступом користувачів.

## Основні можливості:

- **Безпека та відповідність:** Harbor включає потужні функції, такі як сканування вразливостей, підписування образів за допомогою Notary та реплікація образів на основі політик. Це забезпечує безпеку образів і відповідність регуляторним стандартам.
- **Контроль доступу на основі ролей (RBAC):** Дозволяє детально налаштовувати доступ для користувачів і команд, гарантуючи, що тільки авторизовані особи можуть працювати з певними образами.
- **Підтримка мультиоренди (Multitenancy):** Harbor підтримує кілька користувачів або команд (проектів) для керування та розділення контейнерних образів у межах одного реєстру, підвищуючи контроль і ефективність організації.
- **Реплікація:** Дозволяє реплікувати образи між кількома інстанціями Harbor та іншими реєстрами, що підтримує відновлення після збоїв і підвищує доступність образів у різних географічних локаціях.

# Sonatype Nexus Repository OSS

**Sonatype Nexus Repository OSS (Open Source Software)** — це потужний безкоштовний open-source менеджер репозиторіїв, який дозволяє зберігати, організовувати та керувати програмними артефактами. Підтримує широкий спектр форматів пакетів і широко використовується як для приватних, так і для open-source проєктів.

## Основні можливості:

- **Підтримка різних форматів:** Nexus OSS підтримує численні формати пакетів, включаючи **Docker, Maven, npm, NuGet, PyPI, RubyGems** та інші, що робить його універсальним для команд розробників з різними технологічними стеком.
- **Інтелект компонентів (Component Intelligence):** Інтегрується з **Sonatype OSS Index**, надаючи цінну інформацію про **вразливості компонентів**, збережених у репозиторії.
- **Розширюваність:** Дає можливість **розширювати функціонал через плагіни**, покращуючи можливості системи та інтегруючи її у існуючі робочі процеси розробки.
- **Ефективний пошук та навігація:** Має зручний інтерфейс пошуку, який дозволяє розробникам **швидко знаходити та отримувати потрібні артефакти**, збережені в репозиторії.

# Проект Docker “Registry”



# Що це

Реєстр — це програма на стороні сервера з можливістю масштабування без збереження стану, яка зберігає та дозволяє розповсюджувати зображення Docker. Реєстр є відкритим вихідним кодом під дозвільною ліцензією Apache.

# Навіщо це використовувати

Вам слід використовувати Реєстр, якщо ви хочете:

- суворо контролювати, де зберігаються ваші зображення
- повністю володіти своїм каналом розповсюдження зображень
- інтегрувати зберігання та розповсюдження зображень у свій внутрішній робочий процес розробки

# Альтернативи

Користувачам, яким потрібне готове до використання рішення, яке не потребує обслуговування, пропонується звернутись до Docker Hub, який надає безкоштовний розміщений реєстр, а також додаткові функції (облікові записи організації, автоматизовані збірки тощо).

## Вимоги

Реєстр сумісний із системою Docker версії 1.6.0 або новішої.

# Базові команди

Запустіть свій реєстр

```
docker run -d -p 5000:5000 --name registry registry:2
```

Витягніть (або створіть) якесь зображення з концентратора

```
docker pull ubuntu
```

Позначте зображення тегом, щоб воно вказувало на ваш реєстр

```
docker image tag ubuntu localhost:5000/myfirstimage
```

Підштовхніть його

```
docker push localhost:5000/myfirstimage
```

Потягніть його назад

```
docker pull localhost:5000/myfirstimage
```

Тепер зупиніть свій реєстр і видаліть усі дані

```
docker container stop registry && docker container rm -v registry
```

# Про реєстр

Реєстр — це система зберігання та доставки вмісту, яка містить іменовані образи Docker, доступні в різних варіантах

```
Example: the image distribution/registry , with tags 2.0 and 2.1 .
```

```
Example: docker pull registry-1.docker.io/distribution/registry:2.1 .
```

Саме сховище делеговано драйверам. Драйвером зберігання за замовчуванням є локальна файлова система `posix`, яка підходить для розробки або невеликих розгортань. Також підтримуються додаткові драйвери хмарного сховища, такі як S3, Microsoft Azure, OpenStack Swift і Aliyun OSS. Люди, які хочуть використовувати інші системи зберігання даних, можуть зробити це, написавши власний драйвер, який реалізує Storage API.

# Про реєстр

Оскільки безпека доступу до розміщених зображень має першорядне значення, Реєстр спочатку підтримує TLS і базову автентифікацію.

Репозиторій GitHub реєстру містить додаткову інформацію про розширені методи автентифікації та авторизації. Лише дуже великі або публічні розгортання можуть розширити Реєстр таким чином.

Нарешті, Реєстр поставляється з надійною системою сповіщень, викликом веб-перехоплювачів у відповідь на активність, а також широким журналюванням і звітністю, що в основному корисно для великих установок, які хочуть збирати показники.

# Розуміння іменування зображень

Назви зображень, які використовуються в типових командах докерів, відображають їх походження:

- **docker pull ubuntu** наказує докеру отримати образ під назвою **ubuntu** з офіційного Docker Hub. Це просто ярлик для довшої команди **docker pull docker.io/library/ubuntu**
- **docker pull myregistrydomain:port/foo/bar** вказує докеру зв'язатися з реєстром, розташованим за адресою **myregistrydomain:port**, щоб знайти зображення **foo/bar**

# Випадки використання

Запуск власного реєстру є чудовим рішенням для інтеграції та доповнення вашої системи CI/CD. У типовому робочому процесі фіксація вашої системи контролю версій вихідного коду ініціює збірку вашої системи CI, яка потім надішле новий образ до вашого реєстру, якщо збірка буде успішною. Сповіщення від Реєстру потім ініціює розгортання в проміжному середовищі або повідомляє інші системи про доступність нового образу.

Це також важливий компонент, якщо ви хочете швидко розгорнути новий образ на великому кластері машин.

Нарешті, це найкращий спосіб поширювати зображення в ізольованій мережі.

# Вимоги

Ви абсолютно повинні бути знайомі з Docker, зокрема щодо надсилання та витягування зображень. Ви повинні розуміти різницю між демоном і клієм і принаймні зрозуміти базові поняття про роботу в мережі.

Крім того, хоча просто запустити реєстр досить легко, робота з ним у робочому середовищі вимагає навичок роботи, як і будь-яка інша служба. Очікується, що ви будете знайомі з доступністю та масштабованістю системи, журналюванням і обробкою журналів, системним моніторингом і безпекою 101. Глибоке розуміння http і загального мережевого зв'язку, а також знайомство з golang, безумовно, також корисні для складних операцій або злому.



# Розгорніть сервер реєстру

Перш ніж розгортати реєстр, вам потрібно встановити Docker на хості. Реєстр — це екземпляр образу реєстру, який працює в Docker.

У цьому розділі міститься основна інформація про розгортання та налаштування реєстру. Щоб отримати вичерпний список параметрів конфігурації, див.

# Запустіть локальний реєстр

Використовуйте таку команду, щоб запустити контейнер реєстру:

```
$ docker run -d -p 5000:5000 --restart=always --name registry registry:2
```

Тепер реєстр готовий до використання.

# Скопіюйте образ із Docker Hub у свій реєстр

Ви можете отримати образ із Docker Hub і відправити його до свого реєстру. У наступному прикладі зображення **ubuntu:16.04** отримується з Docker Hub і повторно позначається тегом **my-ubuntu**, а потім надсилається до локального реєстру. Нарешті, зображення **ubuntu:16.04** і **my-ubuntu** видаляються локально, а образ **my-ubuntu** витягується з локального реєстру.

```
$ docker pull ubuntu:16.04
```

1. Витягніть образ **ubuntu:16.04** із Docker Hub.
2. Позначте зображення як **localhost:5000/my-ubuntu**. Це створює додатковий тег для наявного зображення. Коли першою частиною тегу є ім'я хоста та порт, Docker інтерпретує це як розташування реєстру під час надсилання.

```
$ docker tag ubuntu:16.04 localhost:5000/my-ubuntu
```

# Скопіюйте образ із Docker Hub у свій реєстр

3. Надішліть образ до локального реєстру, який працює на localhost:5000:

```
$ docker push localhost:5000/my-ubuntu
```

4. Видаліть локально кешовані зображення **ubuntu:16.04** і **localhost:5000/my-ubuntu**, щоб можна було спробувати отримати зображення з реєстру. Це не видаляє образ **localhost:5000/my-ubuntu** з вашого реєстру.

```
$ docker image remove ubuntu:16.04  
$ docker image remove localhost:5000/my-ubuntu
```

# Скопіюйте образ із Docker Hub у свій реєстр

5. Витягніть образ `localhost:5000/my-ubuntu` з локального реєстру.

```
$ docker pull localhost:5000/my-ubuntu
```

## Зупиніть локальний реєстр

Щоб зупинити реєстр, використовуйте ту саму команду зупинки контейнера докерів, що й для будь-якого іншого контейнера.

```
$ docker container stop registry
```

Щоб видалити контейнер, використовуйте `docker container rm`.

```
$ docker container stop registry && docker container rm -v registry
```

# Базова конфігурація

Щоб налаштувати контейнер, ви можете передати додаткові або змінені параметри команді запуску докера.

## Запустіть реєстр автоматично

Якщо ви хочете використовувати реєстр як частину своєї постійної інфраструктури, вам слід налаштувати його на автоматичний перезапуск під час перезапуску Docker або його завершення. У цьому прикладі використовується прапорець `--restart always` для встановлення політики перезапуску для реєстру.

```
$ docker run -d \  
-p 5000:5000 \  
--restart=always \  
--name registry \  
registry:2
```

# Налаштуйте опублікований порт

Якщо ви вже використовуєте порт 5000 або хочете запустити кілька локальних реєстрів для окремих проблемних областей, ви можете налаштувати параметри порту реєстру. Цей приклад запускає реєстр на порту 5001 і також називає його test-реєстр. Пам'ятайте, що перша частина значення -p – це порт хоста, а друга – порт у контейнері. Усередині контейнера реєстр прослуховує порт 5000 за замовчуванням.

```
$ docker run -d \  
  -p 5001:5000 \  
  --name registry-test \  
  registry:2
```

# Налаштуйте опублікований порт

Якщо ви хочете змінити порт, який реєстр прослуховує всередині контейнера, ви можете використовувати змінну середовища **REGISTRY\_HTTP\_ADDR**, щоб змінити це. Ця команда змушує реєстр прослуховувати порт 5001 у контейнері:

```
$ docker run -d \  
-e REGISTRY_HTTP_ADDR=0.0.0.0:5001 \  
-p 5001:5001 \  
--name registry-test \  
registry:2
```



# Налаштування зберігання: налаштуйте місце зберігання

За замовчуванням ваші дані реєстру зберігаються як том Docker у файловій системі хоста. Якщо ви хочете зберігати вміст свого реєстру в певному місці у файловій системі хоста, наприклад, якщо у вас є SSD або SAN, підключені до певного каталогу, ви можете вирішити використовувати замість цього монтування прив'язки. Прив'язане монтування більшою мірою залежить від макета файлової системи хосту Docker, але більш продуктивне в багатьох ситуаціях. У наступному прикладі bind монтує каталог хоста `/mnt/registry` в контейнер реєстру в `/var/lib/registry/`.

```
$ docker run -d \
  -p 5000:5000 \
  --restart=always \
  --name registry \
  -v /mnt/registry:/var/lib/registry \
  registry:2
```

# Налаштуйте серверну систему зберігання

За замовчуванням реєстр зберігає свої дані в локальній файловій системі, незалежно від того, використовуєте ви монтування прив'язки чи том. Ви можете зберігати дані реєстру в сегменті Amazon S3, Google Cloud Platform або на іншому внутрішньому сховищі за допомогою драйверів сховища.

# Налаштування реєстру

Конфігурація реєстру базується на файлі YAML, описаному нижче. Незважаючи на те, що він поставляється з розумними значеннями за замовчуванням із коробки, ви повинні переглянути його вичерпно, перш ніж перевести ваші системи на виробництво.

## Перевизначити певні параметри конфігурації

У типовому налаштуванні, коли ви запускаєте свій реєстр з офіційного образу, ви можете вказати змінну конфігурації з середовища, передавши аргументи -e до частини запуску докера або з файлу Docker за допомогою інструкції ENV.

# Перевизначити певні параметри конфігурації

Щоб перевизначити параметр конфігурації, створіть змінну середовища з іменем `REGISTRY_variable`, де змінна — це ім'я параметра конфігурації, а `_` (підкреслення) — рівні відступу. Наприклад, ви можете налаштувати кореневий каталог серверної частини сховища файлової системи:

```
storage:
  filesystem:
    rootdirectory: /var/lib/registry
```

Щоб змінити це значення, установіть змінну середовища так:

```
REGISTRY_STORAGE_FILESYSTEM_ROOTDIRECTORY=/somewhere
```

Ця змінна замінює значення `/var/lib/registry` на каталог `/somewhere`.

# Заміна всього файлу конфігурації

Якщо конфігурація за замовчуванням не є надійною основою для вашого використання або якщо у вас виникли проблеми із заміною ключів із середовища, ви можете вказати альтернативний файл конфігурації YAML, змонтувавши його як том у контейнері.

Як правило, створіть новий файл конфігурації з нуля під назвою config.yml, а потім укажіть його в команді запуску докера:

```
$ docker run -d -p 5000:5000 --restart=always --name registry \
-v `pwd`/config.yml:/etc/docker/registry/config.yml \
registry:2
```

# Питання та відповіді